Problem Set 3.1 – Sensor Anomaly Filter

Kshitiz Reddy
M.Tech AIML, School of AI, Bennett University
Coding for AI – Module 1, Week 3

Source Code

```python
# Logic Trace: A Python for-loop is bounded by len() of the sequence it
# iterates over. When a batch is [], range(len(batch)) - or iterating the
# batch directly - has zero elements to step through, so the loop body
# never executes and control falls straight through to the next line.
# There is no manual counter or condition to get stuck on, unlike a
# C-style loop, so an empty batch simply does nothing and the audit moves
# on to the next batch.
#
# Author: Kshitiz

telemetry_stream = [
    [22.5, 23.0, 22.8],
    [25.1, "ERR", 24.9],
    [30.2, 35.5, 40.1],
    [22.0, 22.1, "STOP"],
]

shutdown_triggered = False

for batch_id in range(len(telemetry_stream)):
    batch = telemetry_stream[batch_id]
    previous_value = None
    print(f"--- Auditing Batch {batch_id}: {batch} ---")

    for reading in batch:
        if reading == "STOP":
            print(f"Emergency Shutdown at Batch {batch_id}.")
            shutdown_triggered = True
            break

        if reading == "ERR":
            print(f"Noise ignored at Batch {batch_id} (ERR).")
            continue

        if isinstance(reading, (int, float)):
            if reading > 35.0:
                print(f"Anomaly Detected at Batch {batch_id}: {reading}")

            if previous_value is not None:
                delta = abs(reading - previous_value)
                if delta > 5.0:
                    print(f"Spike Detected at Batch {batch_id}: "
                          f"{previous_value} -> {reading} (Delta {delta:.1f})")

            previous_value = reading

    if shutdown_triggered:
        break

else:
```

```
    print("Audit Complete: No system-wide failures")

# Submission Documentation:
# shutdown_triggered starts False and is only ever set True inside the
# inner loop, when a "STOP" reading is found. The inner loop's break just
# exits that batch's reading loop, so straight after it I check the flag
# and break the outer batch loop too, which is what actually stops the
# whole audit early. Because the outer for-loop's else clause only runs
# when the loop finishes without break, and the outer loop is only broken
# when shutdown_triggered is True, the else block prints "Audit Complete"
# in every run except the one where STOP was actually seen. If STOP never
# appears, shutdown_triggered stays False, the outer loop is never broken,
# and the else fires normally at the end.
```

Trace Log

The script was run against the sample telemetry stream from the problem statement. Output below matches the reference trace given in Section 6 of the assignment.

```
--- Auditing Batch 0: [22.5, 23.0, 22.8] ---
--- Auditing Batch 1: [25.1, 'ERR', 24.9] ---
Noise ignored at Batch 1 (ERR).
--- Auditing Batch 2: [30.2, 35.5, 40.1] ---
Anomaly Detected at Batch 2: 35.5
Spike Detected at Batch 2: 30.2 -> 35.5 (Delta 5.3)
Anomaly Detected at Batch 2: 40.1
--- Auditing Batch 3: [22.0, 22.1, 'STOP'] ---
Emergency Shutdown at Batch 3.
```

No `Audit Complete` line appears because `STOP` was hit in Batch 3, so the outer loop's `else` clause is correctly skipped.

Edge Cases Verified

- **Empty batch** – an empty inner list produces no output for that batch and the audit continues normally; the loop simply has zero elements to step through.
- **All-control batch** (`["ERR", "ERR"]`) – both readings are skipped via `continue`, `previous_value` stays `None`, and no delta comparison is attempted.
- **STOP as first reading** – shutdown logic fires immediately, even though no numeric reading was processed in that batch yet.

Submission Notes

The trickiest part of this problem was making sure `break` inside the inner loop didn't just quietly let the outer loop move on to the next batch. Since `break` only kills the loop it's written in, I used a `shutdown_triggered` flag that gets set the moment `STOP` shows up, then checked right after the inner loop ends so the outer loop can break too. That flag is also what the outer `else` depends on – it only prints `Audit Complete` when the outer loop runs to completion without being broken, which is exactly the case where `STOP` was never seen.

For the bonus Rolling Delta check, `previous_value` is reset to `None` at the top of every batch (inside the outer loop, outside the inner one) so a spike from one batch never bleeds into the next. It only updates after a genuine numeric reading, so an `ERR` in between two real values doesn't reset the delta tracking by accident.